CyberAudit

Admin Portal

Dashboard KPIs · Filters · Request Detail · Status Update · PDF Report · Notifications

Project / Projet	CyberAudit Platform
Frontend	React 19 + Vite + Tailwind CSS v4
Auth required	role === admin (JWT via AuthContext)
Key pages	AdminRequestDetailPage, ReportViewerPage
Key component	AdminDashboard (src/components/admin/)
API base	GET/PATCH/DELETE /api/audits/ : Django REST Framework
PDF engine	WeasyPrint (server-side) via POST /api/audits/:id/generate-report/
Email	EmailJS sendStatusNotification() : fire-and-forget
Date	June 2026 / Juin 2026

1. Admin Portal Architecture

1. Architecture du portail administrateur

The admin portal is accessible to users with role=admin. It shares the same DashboardPage entry point as the client portal but renders AdminDashboard instead of ClientDashboard. Admin-specific pages handle individual request management and PDF report editing.

Le portail admin est accessible aux utilisateurs avec role=admin. Il partage le meme point d'entree DashboardPage que le portail client mais affiche AdminDashboard. Les pages specifiques a l'admin gerent la gestion individuelle des demandes et l'edition des rapports PDF.

File	Type	Role EN / FR
AdminDashboard.jsx	Component	(EN) Overview: 4 KPI StatCards + 3-filter table (status/pack/client) + pagination (PAGE_SIZE=5) / (FR) Vue d'ensemble : 4 StatCards KPI + tableau 3 filtres (statut/pack/client) + pagination (PAGE_SIZE=5).
AdminRequestDetailPage.jsx	Page	(EN) Detail view for one request: client info, status/assignee/notes editor, actions (save, generate PDF, notify, archive), history / (FR) Vue détail d'une demande : infos client, éditeur statut/assignée/notes, actions (sauvegarder, générer PDF, notifier, archiver), historique.
ReportViewerPage.jsx	Page	(EN) Dual-mode: Edit mode (input findings + summary + verdict, live score preview) or Read mode (display completed report + download PDF) / (FR) Double mode : Edition (saisie findings + résume + verdict, aperçu score live) ou Lecture (affichage rapport termine + téléchargement PDF).

Route structure / Structure des routes

/dashboard	DashboardPage → AdminDashboard (user.role === admin)
/admin/request/:reference	(EN) AdminRequestDetailPage, edit status, assignee, notes, trigger actions / AdminRequestDetailPage, modifier statut, assigne, notes, declencher actions
/admin/report/:reference	(EN) ReportViewerPage, edit or view security report with findings / (FR) ReportViewerPage, editer ou consulter le rapport de securite avec les findings

2. AdminDashboard : KPIs, Filters & Pagination

2. AdminDashboard : KPI, filtres et pagination

AdminDashboard fetches all audit requests with a single GET /api/audits/ call. It displays four KPI cards (one per status), three independent filters (status, pack, client search), a paginated table, and two action links per row.

AdminDashboard recupere toutes les demandes d'audit via un seul appel GET /api/audits/. Il affiche quatre cartes KPI (une par statut), trois filtres independants (statut, pack, recherche client), un tableau pagine, et deux liens d'action par ligne.

State variables / Variables d'état

requests	Array<AuditRequest>, (EN) full list fetched on mount, never re-fetched on filter change / (FR) Tableau complet récupère au montage, jamais re-recupere lors d'un changement de filtre.
statusFilter	(EN) Selected status: 'all' | 'pending' | 'in_progress' | 'completed' | 'archived' / (FR) Statut selectionne pour le filtrage.
packFilter	(EN) Selected pack code: 'all' | 'audit' | 'security' | 'protection' | 'premium' / (FR) Code pack sélectionne pour le filtrage.
clientQuery	(EN) Free-text search string matched against r.client_info.company_name (case-insensitive) / (FR) Chaine de recherche libre correspondant a r.client_info.company_name (insensible a la casse).
currentPage	(EN) Current pagination page (1-indexed). Reset to 1 on any filter change / (FR) Page de pagination actuelle (index a partir de 1). Remise a 1 lors de tout changement de filtre.

Data fetch / Chargement des donnees

```
// AdminDashboard.jsx — data fetch on mount
useEffect(() => {
  async function load() {
    const { data } = await api.get("/audits/");
    // API can return direct list or paginated { results: [] }
    // axios interceptor in api.js may have already unwrapped pagination
    const list = Array.isArray(data) ? data : (data.results ?? []);
    setRequests(list);
  }
  load();
}, []);  // empty deps — one-time fetch
```

KPI cards / Cartes KPI

Four StatCard components display live counts computed from the requests array. Each maps to one status value. countByStatus() is a simple Array.filter().length, no extra API call.

Quatre composants StatCard affichent des comptages en direct calcules depuis le tableau requests. Chacun correspond a une valeur de statut. countByStatus() est un simple Array.filter().length, aucun appel API supplementaire.

```
// AdminDashboard.jsx — KPI computation
const countByStatus = (s) => requests.filter((r) => r.status === s).length;

<StatCard label="Pending"     value={countByStatus("pending")}
accentClass="text-amber-500" />
<StatCard label="In Progress" value={countByStatus("in_progress")}
accentClass="text-blue-600"  />
<StatCard label="Completed"   value={countByStatus("completed")}
accentClass="text-green-600" />
<StatCard label="Archived"    value={countByStatus("archived")}
accentClass="text-gray-600"  />
```

3-filter system / Systeme de 3 filtres

All filtering is client-side. The three filters are applied together with AND logic: a request must pass all three to be included in filteredRequests. changeFilter() updates the filter and resets pagination to page 1.

Tout le filtrage est cote client. Les trois filtres sont appliques ensemble avec une logique ET : une demande doit passer les trois pour être incluse dans filteredRequests. changeFilter() met a jour le filtre et remet la pagination a la page 1.

```
// AdminDashboard.jsx — triple AND filter
const filteredRequests = requests.filter((r) => {
  const statusOk = statusFilter === "all" || r.status === statusFilter;
  const packOk   = packFilter   === "all" || r.pack?.code === packFilter;
  const company  = r.client_info?.company_name ?? "";
  const clientOk =
company.toLowerCase().includes(clientQuery.trim().toLowerCase());
  return statusOk && packOk && clientOk;  // AND: all three must pass
});

// resetFilters() clears all three + resets page
function resetFilters() {
  setStatusFilter("all"); setPackFilter("all");
  setClientQuery(""); setCurrentPage(1);
}
```

Pagination (PAGE_SIZE = 5)

```
// AdminDashboard.jsx — pagination logic
const PAGE_SIZE   = 5;
const totalPages  = Math.max(1, Math.ceil(filteredRequests.length / PAGE_SIZE));
const safePage    = Math.min(currentPage, totalPages);  // clamp if filters reduce
pages
const startIndex  = (safePage - 1) * PAGE_SIZE;
const pageRequests = filteredRequests.slice(startIndex, startIndex + PAGE_SIZE);

// Numbered page buttons rendered dynamically:
Array.from({ length: totalPages }, (_, i) => i + 1).map((page) => (
  <button onClick={() => setCurrentPage(page)}>{page}</button>
))
```

Table columns / Colonnes du tableau

Column	Content & source
Case #	(EN) r.reference :unique identifier (e.g. CA-2024-0042) / (FR) Identifiant unique (ex. CA-2024-0042)
Client	(EN) r. client_info?.company_name, nested object, ?? '---' / (FR) fallback Objet imbrique, ?? '---' si absent
Pack	(EN) r. pack?.name, nested pack object r. pack?.name — (FR) objet pack imbriqué
Submitted	r.submitted_at via formatDate() — toLocaleDateString('fr-FR') r.submitted_at via formatDate() — toLocaleDateString('fr-FR')
Status	(EN) <StatusBadge status={r.status} /> colored pill badge / (FR) <StatusBadge status={r.status} /> badge colore
Action	(EN) completed: Link to /admin/report/:reference ('View report'). Others: Link to /admin/request/:reference ('View / Edit') / (FR) completed: Lien vers /admin/report/:reference. Autres: /admin/request/:reference.

3. AdminRequestDetailPage : Status Update & Actions

3. AdminRequestDetailPage : Mise a jour du statut et actions

AdminRequestDetailPage is the per-request management screen. It fetches the request by URL reference, shows client information, and lets the admin update status, assign the request, add internal notes, and trigger four actions: save, generate PDF report, send email notification, archive.

AdminRequestDetailPage est l'écran de gestion par demande. Il récupère la demande par référence URL, affiche les informations client, et permet a l'admin de mettre a jour le statut, assigner la demande, ajouter des notes internes, et déclencher quatre actions : sauvegarder, générer le rapport PDF, envoyer une notification email, archiver.

State variables / Variables d'état

request	(EN) Full AuditRequest object (or null while loading / not found). Fetched by loadRequest() / (FR) Objet AuditRequest complet (ou null en chargement). Recupere par loadRequest().
status	(EN) Current value of the status <select>. Initialized from request.status on load / (FR) Valeur actuelle du <select> statut. Initialisee depuis request.status au chargement.
assignedTo	(EN) Selected assignee from ASSIGNEES list (Karim, Sophie, James, Tommy, or empty) / (FR) Assigne sélectionne depuis la liste ASSIGNEES (ou vide = Unassigned).
internalNotes	(EN) Textarea content for internal admin notes (not visible to client) / (FR) Contenu de la textarea pour les notes internes admin (non visibles par le client).
notice	(EN) Inline feedback message (green on success, error text on failure). / (FR) Message de retour en ligne (vert en succès, texte d'erreur en echec).
saving	(EN) Boolean: true while PATCH request is in flight. Disables Save button / (FR) Boolean : true pendant la requête PATCH. Désactive le bouton Save.

loadRequest() : data fetch

```
// AdminRequestDetailPage.jsx — data fetch by reference
const { reference } = useParams();  // e.g. "CA-2024-0042"

const loadRequest = useCallback(async () => {
  setLoading(true);
  const found = await getRequestByReference(reference);
  // GET /api/audits/requests/?reference=CA-2024-0042
  setRequest(found);
  if (found) {
    setStatus(found.status);
    setAssignedTo(found.assigned_to ?? "");
```

```
      setInternalNotes(found.internal_notes ?? "");
    }
    setLoading(false);
  }, [reference]);

  useEffect(() => { loadRequest(); }, [loadRequest]);
```

Client information panel / Panneau infos client

Company	sanitize(clientInfo.company_name), (EN) from request.client_info nested object sanitize(clientInfo.company_name), / (FR) depuis l'objet imbrique client_info
Contact	(EN) sanitize(first_name) + ' ' + sanitize(last_name) / (FR) sanitize(prenom) + ' ' + sanitize(nom)
Email	(EN) sanitize(clientInfo.email), admin can use this for direct contact (FR) sanitize(clientInfo.email), l'admin peut l'utiliser pour un contact direct
Submitted	formatDateTime(request.submitted_at), fr-FR locale DD/MM/YYYY HH:MM formatDateTime(request.submitted_at) , locale fr-FR JJ/MM/AAAA HH:MM
Pack	(EN) pack.name from request.pack nested object / (FR) pack.name depuis l'objet imbrique request.pack
Client message	(EN) sanitize(request.scope_notes), the audit scope submitted by the client sanitize(request.scope_notes) / (FR) le périmètre d'audit soumis par le client

Status & Actions panel / Panneau Statut et Actions

Status <select>	(EN) 4 options: pending, in_progress, completed, archived. Controls status state variable. (FR) 4 options: pending, in_progress, completed, archived. Controle la variable d'etat status.
Assigned to <select>	(EN) ASSIGNEES = ['', 'Karim', 'Sophie', 'James', 'Tommy']. Empty = Unassigned. (FR) ASSIGNEES = ['', 'Karim', 'Sophie', 'James', 'Tommy']. Vide = Unassigned.
Internal notes	(EN) Textarea, not visible to client. Saved via updateRequest(). Textarea, non visible par le client. (FR) Sauvegarde via updateRequest().
Saved status badge	(EN) StatusBadge with request.status (the server-confirmed value, not the local select). (FR) StatusBadge avec request.status (valeur confirmee cote serveur, pas le select local).

Four action buttons / Quatre boutons d'action

Button	API call	Behaviour EN / FR
Save changes	PATCH /api/audits/:id/	(EN) Sends { status, assigned_to, internal_notes } via updateRequest(). Reloads via loadRequest(). Shows 'Changes saved.' or error. / (FR) Envoie { status, assigned_to, internal_notes }. Recharge. Affiche 'Changes saved.' ou erreur.
Generate PDF report	POST /api/audits/:id/generate-report/	(EN) Calls generateReport(reference). On success, navigates to /admin/report/:reference. Shows notice during generation. Appelle generateReport(reference). / (FR) En succes, navigue vers /admin/report/:reference.
Send notification	EmailJS (fire-and-forget)	(EN) Calls sendStatusNotification({to_email, to_name, reference, new_status, message}) then addRequestHistory(). Errors caught silently. (FR) Appelle sendStatusNotification() puis addRequestHistory(). Erreurs captees silencieusement.
Archive request	DELETE /api/audits/:id/	(EN) Calls archiveRequest(reference), soft DELETE on the backend. Reloads. Shows 'Request archived.' / (FR) Appelle archiveRequest(reference)n soft DELETE cote backend. Recharge. Affiche 'Request archived.'

handleSave() : PATCH flow

```
// AdminRequestDetailPage.jsx — handleSave()
async function handleSave() {
  setSaving(true);
  try {
    await updateRequest(reference, {
      status,
      assigned_to:    assignedTo,
      internal_notes: internalNotes,
    });
    await loadRequest();  // reload from server to confirm saved values
    setNotice("Changes saved.");
  } catch {
    setNotice("Error occurred while saving changes.");
  } finally {
    setSaving(false);
  }
}
```

handleSendNotification() : EmailJS

```
// AdminRequestDetailPage.jsx — email notification
async function handleSendNotification() {
  const STATUS_LABELS = {
    pending: "Pending", in_progress: "In Progress",
    completed: "Completed", archived: "Archived",
  };
  // Fire-and-forget — errors are only logged, never shown to admin
  sendStatusNotification({
    to_email:   clientInfo.email,
    to_name:    clientInfo.first_name,
    reference:  request.reference,
    new_status: STATUS_LABELS[status] ?? status,
    message:    internalNotes,  // internal notes included in email
  }).catch((err) => console.error("[emailService]", err));

  // Log action to history (no-op in current backend, returns refreshed request)
  await addRequestHistory(reference, user.first_name,
    `Notification sent — status: ${STATUS_LABELS[status]}`);
  setNotice("Notification sent to client.");
}
```

History panel / Panneau historique

The history panel shows three timestamps derived from the request object: submitted_at, updated_at, and completed_at (if present). Detailed change history is managed server-side by the Django log system. addRequestHistory() is a no-op on the frontend that simply refreshes the request.

Le panneau historique affiche trois horodatages derives de l'objet request : submitted_at, updated_at, et completed_at (si present). L'historique detaille des changements est gere cote serveur par le systeme de logs Django. addRequestHistory() est un no-op frontend qui rafraichit simplement la demande.

4. ReportViewerPage : PDF Report Editor & Viewer

4. ReportViewerPage : Editeur et visionneuse de rapport PDF

ReportViewerPage is a dual-mode component. In Edit mode (admin + status != completed) the admin inputs findings, summary, and verdict, then generates the PDF. In Read mode (report already generated) the report is displayed with score, grade, and findings, with a PDF download button.

ReportViewerPage est un composant en double mode. En mode Edition (admin + statut != completed) l'admin saisit les findings, le resume et le verdict, puis genere le PDF. En mode Lecture (rapport deja genere) le rapport est affiche avec le score, la note et les findings, avec un bouton de telechargement PDF.

Mode switching / Bascule de mode

```
// ReportViewerPage.jsx — mode logic
const isAdmin = currentUser?.role === "admin";

// On load: decide initial mode
setEditMode(isAdmin && aud?.status !== "completed");
// Edit mode: admin viewing a non-completed request
// Read mode: report already completed, or non-admin

// Admin can switch back to edit from read mode:
{isAdmin && (
  <button onClick={() => setEditMode(true)}>Edit report</button>
)}
```

Security scoring: client-side preview

While in edit mode, a live score preview is computed client-side from the findings array using SEV_WEIGHTS. This is purely indicative, the authoritative score is computed server-side on report generation by the Django/WeasyPrint pipeline.

En mode edition, un apercu du score en direct est calcule cote client depuis le tableau findings avec SEV_WEIGHTS. C'est purement indicatif, le score officiel est calcule cote serveur lors de la génération du rapport par le pipeline Django/WeasyPrint.

```
// ReportViewerPage.jsx — client-side score preview
const SEV_WEIGHTS = { critical: 25, high: 10, medium: 4, low: 1 };
const GRADE_TH    =
[[90,"A"],[80,"B+"],[70,"B"],[55,"C"],[40,"D"],[20,"E"],[0,"F"]];

function previewScore(findings) {
  if (!findings?.length) return 100;  // no findings = perfect score
  const deduction = findings.reduce(
    (sum, f) => sum + (SEV_WEIGHTS[(f.severity || "").toLowerCase()] ?? 1), 0
  );
  return Math.max(0, Math.min(100, 100 - deduction));
}

function previewGrade(score) {
```

```
  for (const [threshold, grade] of GRADE_TH)
    if (score >= threshold) return grade;
  return "F";
}
```

Severity weights / Poids des severites

Critical	High	Medium	Low
-25 points	-10 points	-4 points	-1 point
bg-red-200 text-red-900	bg-red-100 text-red-700	bg-amber-100 text-amber-700	bg-green-100 text-green-700

Grade thresholds / Seuils de note

A	B+	B	C	D	E	F	Color class
>=90	>=80	>=70	>=55	>=40	>=20	>=0	A/B+: green-100 B/C: amber-100 D/E/F: red-100

Edit mode flow : adding findings / Mode edition : ajout de findings

```
// ReportViewerPage.jsx — finding management
const EMPTY_FINDING = { severity: "Medium", asset: "", description: "",
recommendation: "" };
const SEVERITIES    = ["Critical", "High", "Medium", "Low"];

// Add a finding (asset is required)
function addFinding() {
  if (!draft.asset.trim()) { setNotice("The Asset field is required."); return; }
  setFindings([...findings, { ...draft }]);
  setDraft({ ...EMPTY_FINDING });  // reset form
  setNotice("");
}

// Remove by index
function removeFinding(i) {
  setFindings(findings.filter((_, idx) => idx !== i));
}
```

handleGenerate() : PDF generation flow

```
// ReportViewerPage.jsx — generate report + auto-complete
async function handleGenerate() {
  setSubmitting(true);  // shows modal loader with rotating messages
  try {
    const result = await generateReportFromFindings(reference, {
```

```
        summary, verdict, findings
      });
      // POST /api/audits/:id/generate-report/
      // Returns { id, security_score, grade, findings_count }

      // Auto-mark as completed if not already
      if (audit.status !== "completed") {
        await updateRequest(reference, { status: "completed" });
      }
      setNotice(`Report generated (score ${result.security_score}/100, grade
${result.grade}). PDF in progress...`);

      // Reload both report and audit objects
      const [fresh, freshAudit] = await Promise.all([
        getReportByReference(reference),
        getRequestByReference(reference),
      ]);
      setReport(fresh); setAudit(freshAudit);
      setEditMode(false);  // switch to read mode
    } catch (e) {
      setNotice("Error: " + (e.response?.data?.detail || e.message));
    } finally { setSubmitting(false); }
  }
```

Generation modal loader / Modal de chargement

While submitting is true, a fullscreen overlay modal is displayed with a spinning indicator and rotating status messages. The messages cycle every 600ms via setInterval(). The rotation stops automatically when the Promise resolves or rejects.

Pendant que submitting est true, un modal en superposition plein ecran est affiche avec un indicateur de rotation et des messages de statut cycliques. Les messages changent toutes les 600ms via setInterval(). La rotation s'arrete automatiquement quand la Promise se resout ou se rejette.

Computing security score...	Step 1: initial message shown immediately via tick()
Saving vulnerabilities...	Step 2: storing findings in database
Rendering HTML report...	Step 3: Django Jinja2 template rendering
Generating PDF with WeasyPrint...	Step 4: WeasyPrint HTML-to-PDF conversion
Finalizing...	Step 5: storing PDF blob, updating audit record

XSS protection in read mode : DOMPurify

ReportViewerPage uses DOMPurify.sanitize() instead of the app-level sanitize() utility. It is configured with ALLOWED_TAGS: [] and ALLOWED_ATTR: [] to strip all HTML tags, only plain text is allowed through.

ReportViewerPage utilise DOMPurify.sanitize() au lieu de l'utilitaire sanitize() de l'app. Il est configure avec ALLOWED_TAGS: [] et ALLOWED_ATTR: [] pour supprimer tous les tags HTML, seul le texte brut est autorise.

```
// ReportViewerPage.jsx — DOMPurify XSS protection
import DOMPurify from "dompurify";

function sanitize(text) {
  return DOMPurify.sanitize(text || "", {
    ALLOWED_TAGS: [],   // strip ALL HTML tags
    ALLOWED_ATTR: [],   // strip ALL attributes
  });
}
// Used on: report.verdict, report.summary, report.findings[].description
// ClientDashboard uses a simpler util/sanitize.js function
```

PDF download in read mode / Telechargement PDF en mode lecture

```
// ReportViewerPage.jsx — PDF blob download
async function handleDownloadPdf() {
  const response = await api.get(`/audits/${audit.id}/report/`, {
    responseType: "blob",
    validateStatus: (s) => s < 500,
  });
  if (response.status === 202) {
    setNotice("PDF being generated. Retry in a few seconds."); return;
  }
  if (response.status >= 400) {
    setNotice("No PDF available."); return;
  }
  // HTTP 200 — trigger browser download
  const url = URL.createObjectURL(new Blob([response.data], { type:
"application/pdf" }));
  const a = document.createElement("a");
  a.href = url; a.download = `rapport-${reference}.pdf`;
  document.body.appendChild(a); a.click();
  document.body.removeChild(a); URL.revokeObjectURL(url);
}
```

5. Admin API Functions : dataService.js

5. Fonctions API admin : dataService.js

Function	HTTP call	Notes EN / FR
updateRequest(ref, changes)	PATCH /audits/:id/	(EN) Looks up request ID by reference first, then PATCHes only changed fields. Merges status, assigned_to, internal_notes / (FR) Recherche d'abord l'ID par référence, puis PATCH uniquement les champs changes.
archiveRequest(ref)	DELETE /audits/:id/	(EN)Soft delete on the Django backend. Fetches ID by reference first / (FR) Suppression logique cote Django. Récupère l'ID par référence d'abord.
generateReport(ref)	POST /audits/:id/generate-report/	(EN) Simple trigger: calls the endpoint, backend uses WeasyPrint. Used from AdminRequestDetailPage basic flow / (FR) Declencheur simple : appelle l'endpoint, le backend utilise WeasyPrint.
generateReportFromFindings(ref, {summary, verdict, findings})	POST /audits/:id/generate-report/	(EN) Full flow: sends findings array + summary + verdict. Server computes score and grade, renders HTML with Jinja2, converts to PDF with WeasyPrint. Returns { security_score, grade, findings_count }. Flux complet : envoie le tableau findings + resume + verdict / (FR) Le serveur calcule le score et la note, rend le HTML avec Jinja2, convertit en PDF avec WeasyPrint.
addRequestHistory(ref)	No-op : returns request	(EN) History is managed server-side by Django's auto-log. This function is a no-op that returns the refreshed request object / (FR) L'historique est gère cote serveur par le log auto de Django. Cette fonction est un no-op.
sendStatusNotification()	EmailJS API (external)	(EN) Sends email via EmailJS template. Parameters: to_email, to_name, reference, new_status, message. Fire-and-forget pattern with .catch(). Email via EmailJS. (FR) Parametres: to_email, to_name, reference, new_status, message. Pattern fire-and-forget.

6. Summary

6. Resume

Concept	Where	Key detail EN / FR
KPI cards	AdminDashboard	4 StatCards: pending (amber), in_progress (blue), completed (green), archived (gray). All from countByStatus().
3-filter system	AdminDashboard	Status (select) + Pack (select) + Client (text search). AND logic. changeFilter() resets to page 1.
Pagination	AdminDashboard	PAGE_SIZE=5. safePage clamps against totalPages. Numbered buttons render dynamically.
Status update	AdminRequestDetailPage	PATCH /audits/:id/ with { status, assigned_to, internal_notes }. Reload confirms saved values.
Assignees	AdminRequestDetailPage	Hardcoded ASSIGNEES = ['', 'Karim', 'Sophie', 'James', 'Tommy']. Empty = Unassigned.
Email notification	AdminRequestDetailPage	sendStatusNotification() fire-and-forget via EmailJS. Errors logged only. Includes internalNotes in message.
Archive (soft delete)	AdminRequestDetailPage	DELETE /audits/:id/, backend soft-deletes. Status becomes 'archived' in UI via reload.
Report edit mode	ReportViewerPage	isAdmin && status != completed. Admin inputs findings, summary, verdict. Live score preview (indicative only).
Score + Grade	ReportViewerPage	Client preview: 100 - sum(SEV_WEIGHTS). Server-authoritative on generation. 7 grades A to F.
PDF generation	ReportViewerPage	POST /generate-report/ sends findings. Django renders Jinja2 HTML, WeasyPrint converts to PDF. Auto-marks completed.
PDF download	ReportViewerPage	GET /audits/:id/report/ as blob. 202 = still generating, 200 = download, 4xx = unavailable.
XSS protection	Both pages	AdminRequestDetailPage: sanitize() util. ReportViewerPage: DOMPurify with ALLOWED_TAGS: [] to strip all HTML.

Admin data flow: DashboardPage (role check) -> AdminDashboard (list, filter, paginate) -> AdminRequestDetailPage (edit one request) -> ReportViewerPage (create/view PDF report). All API calls go through dataService.js functions, which resolve IDs from reference slugs before calling the Django REST endpoints.

Flux de donnees admin : *DashboardPage (verification role) -> AdminDashboard (liste, filtre, pagination) -> AdminRequestDetailPage (editer une demande) -> ReportViewerPage (creer/consulter le rapport PDF). Tous les appels API passent par les fonctions dataService.js, qui resolvent les ID depuis les slugs reference avant d'appeler les endpoints Django REST.*